MÁSTER UNIVERSITARIO EN APRENDIZAJE AUTOMÁTICO Y DATOS MASIVOS · ETSISI - UPM · SCIENTIFIC WORKSHOP

Memoria de Análisis Científico y Guion de Defensa Oral

Artículo seleccionado: **Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes (Sistema EVAPORATE)** — PVLDB Vol. 17, No. 2 (2023)

1. Ficha de Identificación y Validación para Moodle (Apartado 1)

Esta tabla contiene todos los metadatos y enlaces verificados necesarios para registrar el artículo en Moodle y acreditar que cumple holgadamente los requisitos de calidad científica de la guía del Workshop (diapositivas 5–6 de `4_Workshop.pdf`):

Campo Requerido	Información Verificada del Artículo
Título del Artículo	*Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes*
Autores y Filiación	Simran Arora, Brandon Yang*, Sabri Eyuboglu*, Avanika Narayan, Andrew Hojel (*Stanford University*), Immanuel Trummer (*Cornell University*) y Christopher Ré (*Stanford University*).
Revista / Congreso y Año	**Proceedings of the VLDB Endowment (PVLDB)**, Vol. 17, No. 2, págs. 92–105, Octubre/Noviembre 2023 (presentado en la conferencia internacional *VLDB 2024*). ISSN: `2150-8097`.
Referencia Bibliográfica Completa (Formato IEEE/ACM)	S. Arora, B. Yang, S. Eyuboglu, A. Narayan, A. Hojel, I. Trummer, and C. Ré, "Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes," *Proceedings of the VLDB Endowment*, vol. 17, no. 2, pp. 92–105, 2023, doi: 10.14778/3626292.3626294.
Indicadores de Calidad (Requisito Guía Workshop)	• **Revista JCR Q1** en *Computer Science, Information Systems*. • **Congreso CORE A*** (*Very Large Data Bases - VLDB*, máxima categoría mundial en sistemas de datos). • **Antigüedad < 5 años:** Publicado a finales de 2023. • **Formato Research Paper Completo:** 14 páginas a doble columna (límite estricto de PVLDB para artículos principales de investigación: 12 páginas técnicas + 2 páginas con 119 referencias).
Enlaces Oficiales Clicables	• **DOI Oficial:** https://doi.org/10.14778/3626292.3626294 • **PDF Open Access (VLDB):** https://www.vldb.org/pvldb/vol17/p92-arora.pdf • **Preprint arXiv:** https://arxiv.org/abs/2304.09433 • **Código Fuente Oficial (GitHub):** https://github.com/HazyResearch/evaporate
Bloque de la Asignatura	**Bloque I — Almacenamiento de Datos (Fuentes de Datos, Web Scraping e Ingesta ETL/ELT en Data Lakes)**, con conexión transversal directa con el Bloque II (Chunking, TF-IDF y LLMs) y Bloque III (Descubrimiento de Esquemas).

2. Motivación y Problema de Investigación (Apartado 2)

2.1. El problema real en los Data Lakes modernos

En la arquitectura de datos de cualquier organización moderna (hospitales, agencias reguladoras, comercio electrónico, finanzas), los **Data Lakes** almacenan millones de documentos crudos heterogéneos: páginas web en HTML, informes técnicos o clínicos en PDF y correos o registros en texto plano (TXT). Aunque estos documentos contienen información altamente valiosa, **no pueden consultarse mediante SQL ni integrarse en pipelines OLAP/Data Warehouse** mientras permanezcan desestructurados.

> **Formulación matemática del problema (Sección 2 del paper):** Dada una colección de n documentos heterogéneos sin esquema previo $D = \{d_1, d_2, \ldots, d_n\}$, el objetivo es generar automáticamente una vista relacional estructurada compuesta por:
>
> 1. Un **esquema de atributos** $A = \{a_1, a_2, \ldots, a_m\}$ relevante para los documentos.
> 2. Un conjunto de n **registros estructurados (tuplas)** $R = \{r_1, r_2, \ldots, r_n\}$, donde cada registro r_i contiene los valores extraídos del documento d_i para cada uno de los m atributos de A.

2.2. El Trilema de los Sistemas de Extracción y por qué fracasa el Estado del Arte

Los autores demuestran que todo sistema de estructuración de Data Lakes debe equilibrar tres propiedades en tensión (el **Trilema Generalidad – Coste – Calidad**) y que las familias de sistemas previas fracasan al menos en una de ellas:

- **Sistemas específicos de formato Web/DOM (`DOM-LM`, `Ceres`, `ZeroShotCeres`):** Asumen que todos los documentos son páginas HTML con árboles DOM bien formados y que los valores buscados residen exclusivamente en nodos hoja del DOM. Además, exigen entrenar modelos supervisados para cada vertical (películas, universidades, etc.). *Fallo:* Son incapaces de procesar PDFs médicos, textos planos o páginas web como Wikipedia donde un atributo combina varios nodos e hipervínculos dentro de una celda.
- **Sistemas clásicos de Procesamiento del Lenguaje Natural / OpenIE (`OpenIE6`, `Stanford OpenIE`):** Extraen tripletas `(sujeto, relación, objeto)` apoyándose en analizadores sintácticos (POS tagging, NER, árboles de dependencias). *Fallo:* Su precisión se desploma en documentos largos o con tablas/viñetas (como los informes FDA de 20 páginas) y producen esquemas caóticos sin normalizar (en un solo documento FDA, `OpenIE6` extrae **427 relaciones distintas** redundantes).
- **Pipelines ETL manuales programados por ingenieros:** Ofrecen alta calidad y bajo coste de ejecución en CPU, pero exigen cientos de horas de ingeniería humana para escribir reglas con `BeautifulSoup`, `XPath` o expresiones regulares por cada fuente de datos.

3. Propuesta y Contribución Científica: EVAPORATE (Apartado 3)

3.1. Exploración del espacio de diseño con LLMs: Direct vs. Code vs. Code+

Los autores proponen aprovechar la capacidad de *In-Context Learning* (aprendizaje en contexto sin actualizar pesos) de los LLMs modernos utilizando únicamente **6 plantillas de prompt agnósticas al dominio** (Figura 2 del paper). A partir de esta idea, diseñan y comparan tres arquitecturas:

Sistema	Funcionamiento Técnico	Complejidad / Coste LLM	Calidad (Pair F1)	Diagnóstico de los Autores
1. Evaporate-Direct	Divide cada documento d_i en fragmentos (*chunks*) y pasa todos los documentos por el LLM usando los Prompts 1 y 2a para extraer directamente pares `<atributo>: <valor>`.	**Lineal *O(n)*** respecto al nº de documentos. (258M tokens / **$5.157** por 10k docs; **$1,1M** en Wikipedia).	56.6 F1	**Inviable a escala e inconsistente:** el LLM olvida atributos entre documentos, alucina valores y renombra columnas.
2. Evaporate-Code	Toma una muestra $\tilde{D}$ de solo $k = 10$ documentos, identifica el esquema y pide al LLM (Prompt 2b) que escriba **1 función Python por atributo**. Luego ejecuta esa función en CPU sobre los n documentos.	**Sublineal *O(m · k)***, constante respecto a n. (Coste casi nulo por documento adicional).	42.8 F1 (-13.8 pts vs Direct)	**Barato pero frágil:** el 51% de las funciones generadas por el LLM obtiene <50 F1 (y el 40% obtiene <25 F1) por errores de sintaxis o casos borde.
3. Evaporate-Code+	**Genera un conjunto diverso de múltiples funciones Python por atributo y agrega sus predicciones sin etiquetas humanas mediante el Algoritmo 1 (Weak Supervision).**	**Sublineal *O(m · k)*. (1,9M tokens / $39 por 10k docs: ahorro de 110× a 378×).**	**66.7 F1 (+10.1 pts vs Direct y +25.1 pts vs Code)**	**Supera al propio LLM directo en calidad y reduce el coste en dos órdenes de magnitud.**

3.2. Arquitectura detallada de EVAPORATE-CODE+ en 3 Etapas

1. **Etapa 1 — Descubrimiento y Ranking del Esquema (*Schema Synthesis*):**
   - Toma una muestra aleatoria $\tilde{D} \subset D$ de tamaño $k = 10$ documentos, los divide en *chunks* y aplica el `Prompt 1` para extraer atributos candidatos mencionados explícitamente.
   - Normaliza los nombres de atributos (minúsculas, eliminación de puntuación) y los ordena por frecuencia de aparición en la muestra, priorizando los atributos de alta cobertura (*head attributes*) frente a los raros (*tail attributes*).
2. **Etapa 2 — Síntesis de Código Diverso (*Diverse Code Synthesis*):**
   - Para cada atributo $a \in A$, recupera pasajes de la muestra $\tilde{D}$ que contienen la palabra clave del atributo y solicita al LLM que escriba funciones extractoras en Python `def get_attribute(doc: str) -> str`.
   - Para garantizar **diversidad algorítmica** entre las funciones candidatas $F = \{f_1, \ldots, f_k\}$, combina dos plantillas de prompt complementarias sobre distintos subconjuntos de pasajes:
     - `Plantilla P_A` (*Zero-shot regex-oriented*): instruye al LLM para escribir una función Python sin darle ejemplos prefijados, lo que induce soluciones basadas en manipulación de cadenas y expresiones regulares (`re`).
     - `Plantilla P_B` (*Few-shot library-oriented*): incluye 2 ejemplos en contexto que importan librerías de análisis como `from bs4 import BeautifulSoup`, induciendo parsers estructurales sobre el árbol HTML/XML.
3. **Etapa 3 — Agregación de Funciones mediante Supervisión Débil (*Function Aggregation - Algoritmo 1*):**
   - Ejecuta todas las funciones candidatas en CPU (a una velocidad de **0,00025 segundos por documento**) y combina sus salidas usando el **Algoritmo 1** para descartar funciones defectuosas y resolver desacuerdos.

3.3. Innovación Algorítmica: El Algoritmo 1 de Supervisión Débil para Código Generado por IA

En los sistemas clásicos de *Weak Supervision* como **Snorkel** (Ratner et al., VLDB 2017), los ingenieros humanos escriben funciones de etiquetado (*labeling functions*) que cumplen tres supuestos: (1) tienen precisión superior al azar (>50%), (2) devuelven un símbolo explícito de `ABSTAIN` cuando no saben responder, y (3) votan sobre un conjunto pequeño y finito de clases discretas. Sin embargo, las funciones generadas automáticamente por un LLM violan los tres supuestos. El **Algoritmo 1 de EVAPORATE** resuelve este problema sobre una pequeña muestra de validación *D_eval* ($|D_eval| \leq 10$ documentos) tomando las extracciones directas del LLM $\mathscr{F}$ en esos 10 documentos como referencia (*pseudo-ground-truth*):

1. **Distinción entre Abstención y Valor Ausente (Tasa de presencia *e*):** Cuando una función Python devuelve una cadena vacía `""` o lanza una excepción, ¿significa que la función falló (abstención) o que el documento realmente no tiene ese atributo (valor nulo)? El Algoritmo 1 calcula la fracción e de documentos en *D_eval* donde el LLM directo $\mathscr{F}$ extrajo un valor no vacío. Si $e > \tau$ (el atributo está presente en casi todos los documentos), una respuesta vacía de la función se trata como **abstención**; si $e \leq \tau$, se trata como predicción válida de "atributo ausente".
2. **Puntuación y Filtrado de Funciones de Baja Calidad (*Filtering*):** Para cada función candidata f_j, calcula su puntuación empírica $\hat{a}_j$ (Text F1 medio frente a la extracción directa del LLM $\mathscr{F}$ en *D_eval*, penalizando con un término ε las funciones que se abstienen en todos los documentos o repiten una constante). A continuación, **descarta todas las funciones con $\hat{a}_j \leq 0.5$**.

   > **Garantía Teórica — Proposición 1 (Sección 3.3.2):** Suponiendo que las extracciones directas del LLM en *D_eval* tienen tasa de error e y que filtramos las funciones manteniendo aquellas con precisión empírica $\hat{a}_i \geq 1/2 + \gamma$ (con margen $\gamma > e + \varepsilon$), basta un tamaño muestral de validación de tan solo:
   >
   > $$n \geq \left[1 / (2 \cdot (\gamma - \varepsilon - e)^2) \right] \cdot \log(2m / \delta)$$
   >
   > para garantizar con probabilidad al menos $1 - \delta$ que todas las m funciones seleccionadas superan el umbral del 50% requerido por el modelo gráfico de Supervisión Débil.
3. **Discretización del Espacio Abierto de Texto (*Top-b Bucketing*):** Como las funciones devuelven cadenas de texto arbitrarias (espacio infinito de clases), para cada documento agrupa los valores únicos votados por las funciones activas y conserva los *top-b* valores más votados como clases discretas, convirtiendo los valores residuales en abstenciones.
4. **Inferencia con Modelo Gráfico Latente:** Construye la matriz de votos Ψ de las funciones filtradas F' sobre todos los documentos del Data Lake, aprende sin supervisión la precisión y correlaciones de cada función mediante el

modelo gráfico de *Weak Supervision*, y emite el valor consensuado final $\hat{y}_i$ para cada celda de la tabla.

3.4. Extensión a Data Lakes con Múltiples Fuentes Mezcladas (Sección 4.4 y Figura 5)

¿Qué ocurre si en una misma carpeta del Data Lake hay documentos mezclados de 13 sitios web distintos sin ninguna etiqueta que indique a qué web pertenece cada archivo? Si se sintetiza una única función HTML para todos, fallará porque cada web tiene un DOM diferente. En la **Sección 4.4 (Figura 5)**, los autores añaden un paso previo no supervisado: vectorizan los documentos con **TF-IDF** y aplican **clustering K-Means jerárquico** para descubrir automáticamente las particiones homogéneas del Data Lake antes de ejecutar `Evaporate-Code+` en cada clúster, logrando **76.5 Text F1** (el 99,9% del rendimiento de conocer la fuente exacta de antemano, 76.6 F1).

4. Evaluación Experimental y Resultados (Apartado 4)

4.1. Configuración Experimental

- **16 Benchmarks reales en 5 dominios y 3 formatos:**
  - **SWDE Movies** (8 sitios web de cine en HTML, entre 1.063 y 2.000 páginas por web; ej. IMDb, RottenTomatoes, AllMovie).
  - **SWDE Universities** (5 sitios web de universidades en HTML; ej. USNews, CollegeBoard).
  - **Wikipedia NBA Players** (100 páginas HTML complejas de jugadores NBA con tablas, infoboxes y texto libre; 19 atributos).
  - **Enron Emails** (500.000 correos corporativos en texto plano TXT; 15 atributos de metadatos y cuerpo).
  - **FDA 510(k) Premarket Notification Reviews** (100 informes regulatorios de dispositivos médicos en PDF convertidos a TXT, de hasta ~20 páginas cada uno; 16 atributos técnicos y clínicos).
- **4 Modelos de Lenguaje (LLMs) evaluados:** `text-davinci-003` (175B, modelo base de referencia), `GPT-4`, `Anthropic Claude-v1` y `AI21 Jurassic-2 Jumbo Instruct` (178B).
- **Métricas de evaluación:**
  - **ClosedIE (Esquema dado):** Se proporciona la lista de atributos *A* y se mide **Text F1** (F1 a nivel de tokens entre el valor extraído y el ground-truth).
  - **OpenIE End-to-End (Descubrimiento de esquema + Extracción):** El sistema descubre por sí mismo el esquema *A* y extrae los registros *R*. Se evalúa con **Pair F1** (F1 sobre pares completos `(documento, atributo, valor)`).

4.2. Resultados Principales (Tablas 1, 2, 3 y 4 del Paper)

Benchmark (Formato)	ClosedIE: Code (Text F1)	ClosedIE: Code+ (Text F1)	OpenIE: Direct (Pair F1)	OpenIE: Code (Pair F1)	OpenIE: Code+ (Pair F1)	Coste Direct (10k docs)	Coste Code+ (10k docs)	Reducción de Tokens
FDA 510(k) (PDF → TXT)	62.0	**80.1**	45.5	45.6	**62.8** (+17.3)	$2.900 (145M)	**$38** (1,9M)	**77×**
Enron Emails (TXT)	65.7	**93.3**	**93.8**	56.7	86.9 (-6.9)	$425 (21,2M)	**$12** (0,6M)	**35×**
Wiki NBA (HTML complejo)	57.3	**84.7**	44.8	44.6	**68.2** (+23.4)	$13.000 (650M)	**$60** (3,0M)	**217×**
SWDE Movie (8 webs HTML)	58.9	**79.5**	45.2	35.2	**56.8** (+11.6)	$5.660 (283M)	**$46** (2,3M)	**123×**
SWDE Univ. (5 webs HTML)	48.9	**73.7**	53.8	31.9	**59.0** (+5.2)	$3.800 (190M)	**$38** (1,9M)	**100×**
Promedio Global	**58.6**	**82.3 (+23.7)**	**56.6**	**42.8**	**66.7 (+10.1)**	**$5.157 (258M)**	**$39 (1,9M)**	**110× (378× real)**

- **Superación del Estado del Arte Supervisado (Tabla 2):** En el benchmark estándar SWDE, sin usar ningún ejemplo de entrenamiento, `Evaporate-Code+` supera a los mejores modelos supervisados específicos de HTML (`DOM-LM`, `ZeroShotCeres` y `RoBERTa-Structural`) en un **+6% de media en OpenIE** (56.8 vs 54.1 en Movie; 59.0 vs 55.2 en University) y en un **+10% en ClosedIE** (79.5 vs 71.9 en Movie; 73.7 vs 68.0 en University).
- **Estudio de Ablación del Algoritmo 1 (Tabla 4):** Pasar de elegir 1 función (`Code`: 42.8 Pair F1) a combinar las funciones con voto mayoritario simple (`Majority Vote`: 56.2 F1) mejora el resultado, pero aplicar Supervisión Débil estándar apenas sube a 57.7 F1 porque las funciones con precisión <50% corrompen el modelo gráfico. Al activar el **filtrado de funciones con $\hat{a}_j \leq 0.5$** sube a **64.8 F1 (+7.1 puntos)**, y al añadir el **manejo de abstenciones con la tasa *e*** alcanza **66.7 F1** (con una mejora de hasta +7.8 puntos adicionales en FDA).
- **Generalización entre distintos LLMs (Tabla 1):** El método funciona con cualquier LLM capaz de generar código: en OpenIE, `Evaporate-Code+` alcanza **68.6 Pair F1 con GPT-4**, **66.7 con text-davinci-003**, **66.5 con Claude-v1** y **57.2 con Jurassic-2**.

5. Análisis Crítico: Ventajas, Limitaciones y Cuestiones Abiertas (Apartado 5)

5.1. ¿Por qué sintetizar código (Code+) tiene más calidad que preguntar directamente al LLM (Direct)?

Uno de los hallazgos más sorprendentes del artículo es que `Evaporate-Code+` no solo es 110 veces más barato que `Evaporate-Direct`, sino que además obtiene **+10.1 puntos más de F1 medio** (+23.4 puntos en Wikipedia NBA y +17.3 puntos en informes médicos FDA). Los autores auditan los errores de `Evaporate-Direct` en los informes FDA (Sección 4.2.1) y demuestran tres fallos estructurales del LLM cuando extrae datos documento a documento:

1. **Olvido de atributos (Baja exhaustividad por documento):** Aunque el esquema completo de la FDA tiene 16 atributos, el LLM directo olvida extraer de media **4,4 atributos presentes en cada documento (el 27,5%)**.
2. **Alucinaciones y ruido:** El LLM directo extrae de media **9,7 pares atributo-valor por documento que no están explícitamente mencionados** o son irrelevantes para el esquema.
3. **Inconsistencia de nombres de columna:** El LLM directo renombra el mismo concepto de **4 maneras distintas** entre documentos (ej. `"classification"`, `"device classification"`, `"classification name"`, `"regulatory info"`), rompiendo la integridad de la tabla SQL resultante. Por el contrario, una función Python determinista extrae exactamente la misma columna con **78,5% de Precisión media** (Tabla 1).

5.2. Limitaciones identificadas en el artículo

- **Limitación 1 — Documentos con texto libre y baja regularidad sintáctica (El caso Enron Emails):** En la Tabla 3 se observa que `Enron Emails` es el único dataset donde `Evaporate-Direct` (93.8 F1) supera a `Evaporate-Code+` (86.9 F1). Cuando el dato a extraer no sigue ningún patrón sintáctico, etiqueta HTML, cabecera o expresión regular consistente, sino que requiere comprensión semántica de lenguaje natural libre, las funciones Python pierden cobertura (Recall).
- **Limitación 2 — Escalabilidad en esquemas ultra-anchos ($m > 2.500$ atributos):** Como el coste de `Evaporate-Code+` escala con el número de atributos *m* y no con el número de documentos *n*, el análisis de puntos de corte (*Crossover Points*, Figura 3) muestra que si una colección tiene pocos documentos ($n < 40$) o un esquema gigantesco de más de $m > 2.500$ atributos distintos, `Evaporate-Code+` deja de ahorrar tokens frente a `Evaporate-Direct`.
- **Limitación 3 — Dependencia del LLM como "oráculo" en *D_eval*:** El Algoritmo 1 evalúa y filtra las funciones candidatas comparándolas contra la extracción directa del propio LLM sobre 10 documentos (*D_eval*). Si el LLM falla sistemáticamente al extraer un atributo difícil en esos 10 documentos (tasa de error *e* alta en la Proposición 1), el filtro descartará funciones Python que sí eran correctas o eliminará la columna entera.

5.3. Cuestiones abiertas y propuestas de mejora planteadas por el grupo

1. **Arquitectura Híbrida Adaptativa (Router Code+ / Direct):** ¿Por qué elegir exclusivamente entre código o extracción directa? Proponemos un sistema híbrido donde `Evaporate-Code+` se ejecute primero en CPU sobre todo el Data Lake y, únicamente en aquellas celdas donde el conjunto de funciones se abstenga o tenga alta entropía en el modelo gráfico, se invoque selectivamente a `Evaporate-Direct` (o a un SLM local). Esto resolvería la caída de rendimiento en textos libres como Enron manteniendo un coste casi nulo.
2. **Mantenimiento continuo ante cambios de formato (*Schema / Concept Drift*):** Las páginas web cambian su estructura DOM con el tiempo. Una cuestión abierta clave para producción es monitorizar la tasa de abstención de las funciones Python en streaming para disparar la re-síntesis automática únicamente en los atributos cuya tasa de vacío supere el umbral e aprendido en el Algoritmo 1.
3. **Seguridad en la ejecución de código sintetizado por IA (Sandboxing):** Ejecutar código arbitrario generado por un LLM sobre millones de documentos conlleva riesgos de bucles infinitos (*ReDoS* en expresiones regulares catastróficas) o inyección de código, por lo que requiere contenedores aislados con *timeouts* estrictos por función.

6. Relación Detallada con los Contenidos de la Asignatura (Apartado 6)

El artículo seleccionado es un caso de estudio ideal para el *Scientific Workshop* porque integra y extiende conceptos de los **tres bloques de la asignatura**:

Bloque y Documento de Clase	Concepto Estudiado en la Asignatura	Cómo lo Aplica y Amplía el Artículo (EVAPORATE)
Bloque I — Almacenamiento `B1_1_4_Fuentres De Datos.pdf`	• **Diap. 6:** Reto *"From unstructured data to actionable data"* (80% datos no estructurados). • **Diap. 14:** Ejemplo real de informes médicos/FDA difíciles de estructurar. • **Diaps. 20–32:** *Web Scraping* y parsing con `BeautifulSoup`, `lxml/XPath` y expresiones regulares (`re.compile`).	• Evalúa exactamente un benchmark de informes médicos de la **FDA 510(k)** (pasando de 45.5 a 62.8 Pair F1). • Sustituye la programación manual de parsers por **síntesis automática de código Python**: el LLM escribe autónomamente los scripts con `BeautifulSoup` y `re` que en clase programábamos a mano.
Bloque I — Gestión de Información `B2_2_2_Gestión De Informacion.pdf` `B2_2_Warehouse_2025.pdf`	• **Diaps. 9–17:** Pipelines ETL vs ELT en Data Lakes y la limitación de la diap. 14 (*"una función rara vez cubre el 100% de los casos potenciales"*). • **Diaps. 26–27:** Transformaciones *Model-driven / basadas en reglas* vs *Data-driven* (Deep Learning).	• Demuestra empíricamente que el enfoque *Data-driven* puro documento a documento (`Evaporate-Direct`) es inviable a escala ($5.157/10k docs), y que un enfoque híbrido que usa el LLM para generar transformaciones *Model-driven* ejecutables en CPU (`Evaporate-Code+`) reduce el coste 110×. • Resuelve la limitación de la diap. 14 ensamblando múltiples funciones imperfectas con **Supervisión Débil**.
Bloque II — Recuperación de Información y RAG `2_1_1_Indexing_...pdf` `2_2_Semantic_Retrieval_RAG.pdf`	• **Chunking** de documentos largos para ajustar la ventana de contexto del LLM. • Representación léxica **TF-IDF** y búsqueda por palabras clave. • Métricas **Precision, Recall y F1**. • Alucinaciones y limitaciones de los LLMs.	• Aplica *chunking* y filtrado por palabras clave para seleccionar pasajes informativos antes de sintetizar código. • Usa **TF-IDF + K-Means** (Sección 4.4, Fig. 5) para agrupar documentos de múltiples webs sin supervisión. • Cuantifica las alucinaciones del LLM directo (9,7 atributos inventados/doc en FDA) y las elimina mediante código determinista.
Bloque III — Gobernanza e Interoperabilidad `Block 3.1.pdf` / `[3.2] Block 3.2.pdf`	• Descubrimiento e integración de esquemas (*Schema Discovery & Matching*). • Extracción basada en ontologías/esquemas cerrados vs abiertos.	• Resuelve tanto el escenario de esquema cerrado (**ClosedIE**) como el de descubrimiento automático de esquemas abiertos (**OpenIE**) priorizando atributos de alta cobertura (*head attributes*) y normalizando nombres de columnas.

7. Guion de Exposición Oral para el "Scientific Workshop" (3–4 Estudiantes)

Este guion está diseñado para una presentación ágil, rigurosa y coordinada de **10 minutos** apoyada en el póster científico (`Poster_Cientifico_EVAPORATE.pdf`). Incluye indicaciones tanto para grupos de **4 integrantes** como de **3 integrantes** (si sois 3, el Estudiante 3 asume los bloques 3 y 4 resumidos).

Estudiante 1: Identificación del Artículo, Motivación y Trilema del Estado del Arte

Tiempo: 0:00 – 2:30 (2,5 min)

Dónde señalar en el póster: Cabecera superior y Tarjeta 2 (arriba a la izquierda).

Guion hablado sugerido:

*"Buenos días/tardes a todos. Nuestro grupo ha analizado el artículo **'Language Models Enable Simple Systems for Generating Structured Views of Heterogeneous Data Lakes'**, publicado a finales de 2023 en **PVLDB (Vol. 17)**, revista JCR Q1 y conferencia CORE A*, realizado por investigadores de la Universidad de Stanford (grupo Hazy Research de Christopher Ré) y la Universidad de Cornell.*

El problema que aborda el paper conecta directamente con el Bloque I de nuestra asignatura: hoy en día, los Data Lakes almacenan millones de archivos heterogéneos —páginas web HTML, informes clínicos o regulatorios en PDF de 20 páginas, o correos en texto plano—. Como vimos en clase, el 80% de los datos son no estructurados y no podemos lanzar consultas SQL sobre ellos sin transformarlos antes en una tabla relacional con un esquema de atributos A y registros R.

*Los autores plantean que cualquier sistema de extracción se enfrenta a un **trilema entre Generalidad, Coste y Calidad**: por un lado, los modelos del estado del arte web como `DOM-LM` o `Ceres` exigen entrenamiento supervisado en cada dominio y solo funcionan en árboles HTML DOM, siendo inútiles en PDFs o texto plano. Por otro lado, los sistemas NLP de extracción abierta como `OpenIE6` se rompen en documentos largos y generan cientos de relaciones sin normalizar (427 relaciones distintas en un solo informe de la FDA). Y programar scrapers manuales con BeautifulSoup o regex para cada fuente no escala."*

Estudiante 2: La Propuesta (Direct vs. Code vs. Code+) y el Algoritmo 1 de Weak Supervision

Tiempo: 2:30 – 5:30 (3,0 min)

Dónde señalar en el póster: Tarjeta central 3 (Tabla comparativa de las 3 arquitecturas, diagrama de 3 etapas y Algoritmo 1).

Guion hablado sugerido:

*"Para resolver este trilema sin entrenar modelos específicos, los autores exploran el aprendizaje en contexto de los LLMs mediante el sistema **EVAPORATE** y comparan tres arquitecturas que revelan un compromiso muy interesante entre coste y calidad:*

*La primera opción, **Evaporate-Direct**, consiste en pasar todos los documentos del Data Lake por el LLM para que extraiga directamente los pares atributo-valor. ¿El problema? Su coste crece linealmente O(n) con el tamaño del Data Lake: procesar 10.000 documentos cuesta más de 5.100 dólares (258 millones de tokens), y procesar Wikipedia entera costaría 1,1 millones de dólares.*

*La segunda opción, **Evaporate-Code**, muestrea solo k = 10 documentos y le pide al LLM que programe una función Python en BeautifulSoup o regex por cada atributo para luego ejecutarla gratis en CPU. El coste cae drásticamente, pero la calidad se desploma casi 14 puntos F1 porque el 51% de las funciones generadas por el LLM tiene una calidad inferior a 50 F1.*

*Aquí nace la contribución principal del paper: **Evaporate-Code+**. En lugar de confiar en una sola función por atributo, el sistema usa varias plantillas de prompt para sintetizar un conjunto diverso de múltiples funciones Python candidatas y agrega sus salidas mediante el **Algoritmo 1 de Supervisión Débil (Weak Supervision)**. Como la teoría clásica de Weak Supervision exige que las funciones tengan precisión mayor del 50% y clases discretas, el Algoritmo 1 evalúa las funciones sobre solo 10 documentos comparándolas con el LLM, estima la tasa de presencia del atributo para distinguir cuándo una función se abstiene de cuándo el dato realmente no existe, **filtra todas las funciones con puntuación menor o igual a 0,5** (respaldado teóricamente por la Proposición 1 del paper) y agrupa las cadenas de texto en los top-b buckets más frecuentes antes de resolver el modelo gráfico."*

Estudiante 3: Evaluación Experimental y Análisis Crítico

Tiempo: 5:30 – 8:00 (2,5 min)

Dónde señalar en el póster: Tarjetas 4 y 5 (columna derecha: KPIs verdes, Tabla 3 de resultados, gráfica de barras y caja de análisis crítico).

Guion hablado sugerido:

"En la evaluación experimental sobre 16 benchmarks reales de 5 dominios y 3 formatos (HTML, PDF y TXT) con 4 LLMs distintos, los resultados son contundentes: ***Evaporate-Code+ reduce el consumo de tokens en 110 veces de media*** *(y 378 veces en el tamaño real de los benchmarks, bajando de 5.157$ a solo 39$ por cada 10.000 documentos), ejecutándose en CPU en solo 0,25 milisegundos por documento.*

Pero lo más sorprendente es que ***Code+ supera en calidad al propio LLM directo en +10,1 puntos Pair F1 (+18%)****, y supera además al estado del arte supervisado en HTML como DOM-LM sin usar un solo dato de entrenamiento. ¿Por qué un conjunto de scripts Python generados por un LLM es más preciso que el propio LLM leyendo cada documento? Al auditar los informes médicos de la FDA, se demuestra que el LLM directo olvida el 27,5% de los atributos por documento, alucina 9,7 valores inexistentes y renombra la misma columna de 4 formas distintas, mientras que las funciones Python imponen consistencia determinista en toda la tabla.*

Como análisis crítico, hemos identificado dos limitaciones clave: primero, en textos libres sin estructura sintáctica repetitiva —como el cuerpo de los correos de Enron—, Evaporate-Direct sigue superando a Code+ (93,8 frente a 86,9 F1), porque las expresiones regulares no capturan semántica libre. Segundo, el coste de Code+ escala con el número de atributos m: si un esquema tuviera más de 2.500 columnas o menos de 40 documentos, dejaría de ser rentable."

Estudiante 4 (o cierre compartido): Conexión con la Asignatura y Propuestas de Mejora

Tiempo: 8:00 – 10:00 (2,0 min)

Dónde señalar en el póster: Tarjeta 6 (abajo a la izquierda) y cuadro amarillo de cuestiones abiertas (abajo a la derecha).

Guion hablado sugerido:

"Para concluir, este trabajo conecta los tres bloques de nuestra asignatura: del ***Bloque I****, automatiza los parsers de Web Scraping en BeautifulSoup, XPath y regex que vimos en el tema de Fuentes de Datos, y demuestra en el tema de ETL/ELT cómo combinar transformaciones Data-driven y Model-driven para superar la limitación que vimos en la diapositiva 14 ('una sola función rara vez cubre el 100% de los casos'). Del* ***Bloque II****, aplica chunking, recuperación léxica y clustering TF-IDF + K-Means para separar documentos mezclados de múltiples webs sin supervisión; y del* ***Bloque III****, resuelve el descubrimiento automático de esquemas.*

Como propuesta de mejora del grupo, planteamos una ***arquitectura híbrida adaptativa****: ejecutar Evaporate-Code+ por defecto en CPU y llamar a Evaporate-Direct únicamente en aquellas celdas concretas donde las funciones Python se abstengan, combinando así el bajo coste del código con la comprensión semántica del LLM. Muchas gracias y quedamos a vuestra disposición para cualquier pregunta."*

8. Batería de Preguntas Posibles del Tribunal / Profesores y Respuestas (Q&A)

Pregunta 1: "¿Cómo es posible que las funciones Python generadas por el LLM (Evaporate-Code+) den mejor F1 que usar el propio LLM directamente en cada documento (Evaporate-Direct), si las funciones las ha escrito ese mismo LLM?"

Respuesta: Por dos razones demostradas en la Sección 4.2.1 del paper: primero, la **consistencia global**. Cuando `Evaporate-Direct` procesa 10.000 documentos uno por uno en llamadas independientes, sufre de olvidos (en FDA omite el 27,5% de los atributos presentes por documento), alucinaciones (inventa 9,7 atributos por informe) y falta de normalización (llama al mismo atributo de 4 formas distintas). En cambio, en `Evaporate-Code+` el LLM se concentra en los fragmentos donde el atributo sí aparece claramente, sintetiza funciones deterministas que aplican exactamente la misma regla a los 10.000 documentos, y el Algoritmo 1 filtra el 51% de funciones ruidosas quedándose solo con el código de alta precisión (78,5% de Precisión en Tabla 1).

Pregunta 2: "¿Qué pasa si en el Data Lake tenemos archivos mezclados de muchas páginas web distintas y no sabemos de qué web viene cada archivo? ¿No fallarán las funciones de BeautifulSoup?"

Respuesta: Excelente pregunta; los autores lo resuelven específicamente en la **Sección 4.4 (Figura 5 del paper)**. Si se entrena un único conjunto de funciones sobre 13 webs mezcladas sin agrupar, el rendimiento cae a 50,3 Text F1 porque el árbol DOM cambia de una web a otra. Para solucionarlo sin etiquetas humanas, el sistema vectoriza primero los documentos usando **TF-IDF** y aplica **K-Means jerárquico** para descubrir automáticamente los clústeres de documentos con estructura similar. Al sintetizar funciones por clúster, recupera un **76,5 Text F1**, prácticamente idéntico al 76,6 F1 de conocer las fuentes reales de antemano.

Pregunta 3: "¿Por qué no basta con hacer un voto por mayoría (Majority Vote) entre las funciones Python generadas en vez de usar el Algoritmo 1 de Weak Supervision?"

Respuesta: En el estudio de ablación de la **Tabla 4**, los autores prueban exactamente *Majority Vote* y obtienen **56.2 Pair F1** frente a los **66.7 Pair F1** del Algoritmo 1 completo (más de 10 puntos de diferencia, y en el dataset de universidades cae de 59.0 a 44.3). La razón es que más de la mitad de las funciones generadas por el LLM son de baja calidad (<50 F1) y muchas cometen errores correlacionados (por ejemplo, devolver cadenas vacías o capturar la etiqueta equivocada). Si todas votan con el mismo peso, las funciones malas ganan la votación; el Algoritmo 1 filtra previamente las funciones con calidad empírica $\hat{a}_j \leq 0.5$ en *D_eval* y pondera estadísticamente la fiabilidad de las supervivientes.

Pregunta 4: "¿En qué casos NO recomendaríais usar Evaporate-Code+ en una empresa?"

Respuesta: Basándonos en la Figura 3 y la Tabla 3 del paper, no lo recomendaríamos en tres escenarios concretos: (1) cuando la colección de documentos es muy pequeña (**menos de $n = 40$ documentos**), porque el coste fijo de sintetizar y validar múltiples funciones supera al de usar `Evaporate-Direct`; (2) cuando el esquema buscado es extremadamente ancho (**más de $m = 2.500$ atributos**); y (3) cuando los documentos son texto narrativo libre sin ninguna estructura sintáctica o léxica recurrente (como el cuerpo de ciertos correos en *Enron Emails*), donde una regla regex o DOM no puede sustituir la inferencia semántica del LLM.